

Composition Problem in Programming Language Java

15.1. THE PROBLEM DESCRIPTION

As is known in programming language Java there is no concept of the pointer. That is all not primitive types (arrays, strings, objects of any class) are realized as pointers, but they appear without specifying, that they are pointers. Thus, to created strings, arrays and all objects created as instances of classes it is possible to address only through pointers. Fields, variables, parameters of methods and created anonymous quantities contains (in the memory allocated under it) either primitive value, or the structure address, on what they refer.

Thereby Java solves the acute problems associated with pointers. The problem of dangling pointers is one of commonly encountered problems in programming languages and demands excessive time in process of debugging. A garbage problem (the object where there is no reference) though not commonly encountered problem, but if it happens a memory overflow in very inconvenient moment the consequence will be pitiable. These problems do not arise in Java because the programmer obviously does not work with pointers. Memory management (garbage collection) – a problem of the program runtime environment and the programmer does not interfere with this process.

But Java elegantly resolving the above-stated problems, complicates to resolve one problem of a design stage - an aggregation problem. In process of designing somehow or other are modelled real entity or processes. Each real entity has corresponding properties and naturally defined operations which apply on entity of given type. If some entity's property is difficult, that is in turn is entity, in programming languages such modelling (and the given relation between these objects) is called as aggregation. In the text it is used the word "object" as some model of entity of the real world. Thus, the object which aggregates - is called whole, and object-property - as a part. For example, the document aggregates its contents, the computer aggregates the device - the processor, the car aggregates the engine etc.

Though the object included in aggregation is understood as the object part, some such objects-parts can physically contain not in the including object. For example, the author of the book is out of the book, the person has the bank account, not including it physically in itself, the shareholder exclusively owns the actions, but they do not enter into it physically, car registration certificate can appear out of the car etc. These relations, undoubtedly, are aggregation relations, but rather conceptual, than physical in nature. The including object still means whole, but its part, contained in whole indirectly.

In languages with pointers [1] the relationship of the whole and its indirect part (the property) is implemented by pointers, the object-part of this relationship is named as a part-pointer or part-link. Properties physically included within the whole implemented directly allocating (expanding) memory for the part of a whole, included in this relationship the object-part is named as a part-value or expanded part. Thereby it is determined the physical boundary of the whole object in these languages. Everything, that within object (within the boundaries), that is physically consisted in it - included (deployed) in it as part of its values, and everything, with whom the object has aggregation, but does not physically consisted into it, that is not contained within the (border) of the object - included inside it as pointers to the corresponding values. In [3] the types of objects, the values of which are deployed inside another object, are called as a expanded types.

In object-oriented design language UML [2], in the diagram of classes, there are two relations of association for modeling of such relations. The aggregation relation is used to specify the reference (not included within the boundaries of the whole) of a relationship with the part. The composition relation is used to indicate the relationship between the whole and its expanded part (see Fig. 15.1) [2].

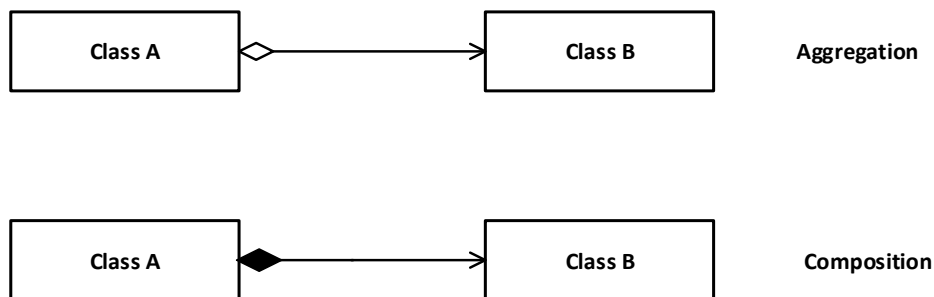


Fig. 15.1. Aggregation by reference (top) and the by value (bottom)

Source: [2]

In fact, instead of a pointer, you can use the record key on this object in the database objects of this class. It is also aggregation, but without a physical inclusion.

The concept of the physical boundaries of the object is necessary in order to determine the perform semantics of some basic operations on the object. These operations are the creation, assignment, copying (cloning), comparison and deletion. For example, if you copy an object, for the parts - values should be copied to these objects-parts (values), and for parts-pointers should be copied pointers to objects-parts, that is should be copied addresses of the parts. In addition the physical boundary is needed to determine whether the object part is shared between multiple objects, or it is only a property of its owner - the object-whole. There are many examples where the parts are divided among several entities - the owners, for example, other book can point to this author, that is, two books have one and the same author, so both books refer to one and the same author. Bank accounts book can point to a given customer account, database in the traffic police to this registration certificate etc.

If the object value changes, the change is reflected in all objects that share this object as a part of them and refer to this object through a pointer. In real life, it really should be. Indeed, if we change the personal data of the author of the book, then is changed the data for all books of this author. Similarly, if you change the balance or the state in the bank account, this is reflected in the customer and the bank's books of accounts. If the registration certificate of the car data is adjusted based traffic police data (for example, after painting), this change is reflected in the car's registration certificate.

Conceptually, the above operations (creation, assignment, ...) on the objects based on two tests: an identity verification and equivalence checking. Checking identity - is to check whether the objects are the same object in the real world. Objects of the same class always differ from each other as the entity. Testing for equivalence - checking on whether the objects have the same state [3].

On the assumption of this reason, in languages with pointers above above-mentioned operations on objects are defined by standard operations naturally, within the limits of physical border of this entity. When working with parts-pointers it is necessary keep in mind, that the object-whole and object - part live separately from each other, it is possible to create and to destroy instance of corresponding classes independently, and parts-values "is born" together with whole, "lives" in it, he (whole) unique object which to it has access, and "dies" too together with the whole.

In these languages, at creation of object the object is created within the limits of its physical borders. For parts-values is allocated the place for object of a corresponding class, and for parts-pointers is allocated the place only for the pointer, without dependence to what class this concerns this part. At comparison the entities are compared also within the limits of their physical borders: parts - values are compared as comparison of objects, for a parts - references are compared the addresses (value of the pointers). The assignment operation is implemented also within the limits of physical borders: parts-pointers is copied as value of the pointer, that is is copied the address, and for parts-values is copied the object. Copy operation is implemented also just as assignment operation.

For languages with pointers, along with the above-stated benefit for delimitation an entity borders, the natural approach to aggregation designing, there are problems which need to solve. Except problems of dangling references and garbage there are following moments: to avoid structural dependence through references it is important to adhere to any arrangement concerning creation and destruction of objects, the reference on which can contain in different places. It is necessary, that it someone did one. Besides, it is impossible to refer (to establish aggregation under the reference) from other object on object which is included as a part of other object. At class designing it is debared (direct or indirect) recursive aggregation of objects by value with type of the class.

In Java language the above-stated problems disappear of itself. But a following question arises, very actual to provide exact modelling at designing [3]: how to define entity border in Java? Ex facto the answer - in no way. All properties with primitive type are realised as a field of such type, and all not primitive - as pointers.

In Java the aggregation relation (modelling of parts-pointers) is realised without problems because this relation is based on the pointer. The problem concerns only to the composition relation, to be exact - definition of physical border of modelled object. Further just this relation is considered, that is part-value realisation is considered not by expansion of object-part in whole, but in another way.

Actually, if to look memory structure (to take snapshot) during Java program runtime, looking on object and on its part is no way to distinguish this is a part-value or a part - pointer because all parts are connected with the whole object through pointers.

There is the philosophical point of view [8], based that the relation - primary, entity - secondary. The entity in the real world can present as some relation between infinite quantity of components of this relation. At designing are used real world models and abstract models. From this point of view, for example, the point on a plane is the relation of two real numbers, the book can be modelled as some relation between the name, publisher, the author and a content, the person - as the relation between name, a birthdate, the address of residing and the social identifier, the car - as the relation between the body, the engine, electric system, running gear and

registration certificate. All theory of relational databases (relational data model) [7] is based on it.

It follows that from this philosophical point of view, the physical border specifies not solidity (indivisibility of incorporating parts) of whole object, and is tightly coupling of these incorporating parts, that is with close interaction of parts of this object between itself and with covering whole object. These interactions rather are strong, than interactions between parts of different objects. The interaction interface, supporting these relations is fixed. Thus it is possible to tell about a modular design of the whole. Each part is the module has a interface, responsibility. It is possible to replace a part entering into the whole object with other part if replacing part supports that interface.

15.2. EXPANDED OBJECTS

In languages with pointers there is the concept - expanded objects. This concept is used to model complex objects, parts of which are "deployed" within the physical boundaries. Deployed objects inside of a not fully support modularity - namely, the dynamic interchangeability. The complex object "borned" with deployed inner parts may not replace inner part to another object implementing the same interface in runtime. In the example of C++ (Fig. 15.2), can say that the polymorphism is broken.

```
#include <iostream>
using std::cout;
using namespace std;

class A {
private:
    int x;
public:
    A() {}
    int getX() {return x;}
    void setX(int i) {x=i;}
};

class B : public A {
private:
    int y;
public:
    B() { }
    B(int i) { y = i;}
```

```

        int getY() {return y;}
        void setY(int i) {y=i;}
    };
    class C {
    private:
        A a;
        int i; // some field
    public:
        C() { }
        A* getA() {return &a;};
        void setA(A a1) {a = a1;}
    };
    int main() {
        C c = C();
        B b1;
        b1.setX(10);
        b1.setY(12);
        cout << "Object field's values before polymorphous assignment:" << endl;
        cout << "Value of the b1.x: " << b1.getX() << endl;
        cout << "Value of the b1.y: " << b1.getY() << endl;
        // polymorphous assignment
        c.setA(b1);
        // declaring a pointer and type conversation from A* to B*
        A * a = c.getA();
        B * b2 = (B*)a;
        cout << "Object field's values after polymorphous assignment:" << endl;
        cout << "Value of the b2.x: " << b2->getX() << endl;
        cout << "Value of the b2.y: " << b2->getY() << endl;
        while (true) {};
        return 0;
    }

```

Fig. 15.2. Example 1

Source: own work

The program from Fig. 15.2 gives the following result:

```

Object field's values before polymorphous assignment:
Value of the b1.x:      10
Value of the b1.y:      12
Object field's values after polymorphous assignment:
Value of the b2.x:      10
Value of the b2.y:      65535

```

Here it is declared the three classes: Class **A**, its subclass **B**, and class **C**, having expanded part-value (named **a**). In body of main function it is created an object of class **C** (named **c**). During the creation, inside of this object (of class **C**) it'll be expanded an object of class **A** (field named as **a**). Further, it is created object of class **B** (named **b1**) and values **10** and **12** will be assigned to corresponding fields of **b1**. These three values will be printed on the console. Further, by way of **c.setA(b1)** in

the field **a** (of type **A**) of the object **c** it polymorphic is assigned the object of the class **B** (named **b1**). Through **A* a = c.getA()** will be created the pointer **a** referencing to the field **c.a**. So as the pointer **a** actually references to the object of the class **B**, it can be casted to pointer referencing to object of class **B** (named **b2**). The values of the fields of this object have to be same with values of corresponding fields of **b1**, but, as seen from the program output the value of the field **y** is difference (not **12**, but **65535**).

This situation isn't surprisingly, so as in time of creating the object **c** a memory under this object is allocated taking into account the expanded object **a**. Then, in time of polymorphic assignment of the object **b1** to the object of **a**, the value of the field **y** is not fit in allocated space for the object **a**. Output value **65535** – is result of ordinary trick (weak type control) of C++.

So, at working with expanded objects polymorphism don't work, but modularity, reusability, and object oriented programming in general, needs polymorphism.

Actually, modern aggregated devices are made by modularity principles. For example, a care although contains engine inside of its physical boundaries, it can be replaced engine with another one, for example by more modern one, having addition engine components. It is essential, a new engine must fit into space of the old one and it must be possible to turn on all of interface items of engine with remainder parts of the care. For a computer, replacement of the RAM or disk memory, CPU – everywhere are applied. Today it can be say about artificial organs for man. Above mentioned examples say that physical boundary doesn't mean monolithic nature or wholeness, most likely it means inside of boundary of the devises relationships between items are more closer and external relationship between items of inside physical boundary and external world are weak.

Physical boundary emphasizes, that part is also object, and in principle it can be regarded as independently. Membership of some components of the relationship to the covering object statically nowhere is indicated. This is determined by the semantics of the above mentioned operations of creation, assignment, copying (cloning), comparison and deletion.

As seen above, for languages with pointers correct modeling of the physical boundaries (composition) can't be implemented with expanded parts. It turns out that in Java all well conceived, because the composition is realized on the basis of indicators.

Many books on the Java language [4, 5] avoid this design problem. In books such as [6], dedicated to the design, did not disclose the implementation methods of composition and aggregation. Michelle Yaiser [9] discusses the composition and simple aggregation, based only the creation operation.

15.3. AGGREGATION

Aggregation and composition must be implemented with pointers. When pointer-part is implemented (simple aggregation), it is created separately and is passed by parameter (or in some other way) to constructor during create a whole – see example 2 in Fig. 15.3.

```
class A {
    private int x;
    public A() {}
    public A(int x) {this.x = x;}
    public int getX() {return x;}
    public void setX(int x) {this.x = x;}
};

class B extends A {
    private int y;
    public B() {}
    public B(int y) {this.y = y;}
    public B(int x, int y) {super(x); this.y = y;}
    public int getY() {return y;}
    public void setY(int y) {this.y = y;}
}

class C {
    private A a;
    private int c; // some field
    public C(A a) {this.a = a;}
    public A getA() {return a;}
    public void setA(A a) {this.a = a;}

    public static void main(String[] args) {
        B b1 = new B(10,12);
        C c = new C(b1);
        System.out.println("Object field's values before changing:");
        System.out.println("Value of the x: " + ((B)c.getA()).getX());
    }
}
```

```

        System.out.println("Value of the y: " + ((B)c.getA()).getY());
        System.out.println("Object field's values before changing:");
        b1.setX(24);
        b1.setY(48);
        System.out.println("Object field's values after changing:");
        System.out.println("Value of the x: " + ((B)c.getA()).getX());
        System.out.println("Value of the y: " + ((B)c.getA()).getY());
    }
}

```

Fig. 15.3. Example 2

Source: own work

The program from Fig. 15.3 gives the following result:

```

Object field's values before changing:
Value of the b1.x: 10
Value of the b1.y: 12
Object field's values before changing:
Object field's values after changing:
Value of the b1.x: 24
Value of the b1.y: 48

Object field's values before changing:
Value of the b1.x: 10
Value of the b1.y: 12
Object field's values before changing:
Object field's values after changing:
Value of the b1.x: 24
Value of the b1.y: 48

```

Here, if you look at the class **C** code, you can see that when you create an object of class **C** it is necessary to pass to the constructor an object of class **A**. Object of the class **C** with its field named **a** included in the aggregation relationship (part-pointer). As can be seen from the main code, we can instead of an object of class **A**, polymorphic pass an object of class **B** (named **b1**), because class **B** is a subclass of class **A**. If we change values of the **b1** fields, values of the fields to which we can access through **c.getA()** will be changed also.

15.4. COMPOSITION

As discussed above, in the composition, we need to implement a physical boundary to define the semantics of the operations of creation, assignment, copying (cloning), comparison and deletion. We consider these operations in more detail.

Construction (creating) of composition

In terms of composition, object is "born" with his physical boundary, that is, all its parts are created with the object of covering them. Parts of the whole should not have other links from outside. Composite object can be created by passing to the constructor as a parameter all its parts. Let p1 of type P1, p2 of type P2, ..., pn of type PN - are separate parts of an object of class C and in addition objects of this type have no other part. Then, the template from Fig. 15.4 can be used for the C class constructor.

```
class C {
    private P1 p1;
    private P2 p2;
    ...
    private PN pn;
    public C(P1 p1, P2 p2,..., PN pn) { // Constructor initializing parts
        this.p1 = p1.clone();
        this.p2 = p2.clone();
    ...
        this.pN = pN.clone();
    }
    public C(...,P1 p1, P2 p2,..., PN pn, ...) { // Another constructors
        C(p1,p2,...,pn);
        ...
    }
}
```

Fig. 15.4. Example 3

Source: own work

All constructors must call the class constructor that initializes the parts. To prevent links from other objects there are not set references to the passed parameter objects, instead set a link to the clones of these objects. This constructor assigns clones of the object-parts to the fields of the object. Cloning policy of the objects p1, ..., pn define respective classes P1, ..., PN.

Comparison

Assignment operations, comparison and copying (cloning) are binary operations and in the operation involved two objects.

Operation comparison's result has **boolean** type. Thus, this operation is a binary equivalence relation.

Programming languages for data comparison commonly use the operation "==" (or "=" in Pascal like languages). In languages with pointers by this operation are compared all of the values for the parts-values, for the parts-pointers compares the values of these pointers. As previously mentioned, in this manner is determined the physical boundary of the object. In C ++ the concept of string (in its physical boundary) includes line contents, therefore, due to the fact that the strings in C ++ implemented as a pointer, operation "==" is overridden and compares the contents of the string, and in the C language for this aim is used **strcmp** function. In the Java language this operation is used to compare the primitive data objects For objects, using of of this operation has the semantics of comparing the address of objects, rather than their value. To compare strings (**String** type objects) is used methods **equals**, **equalsIgnoreCase** and **compareTo**:

- **boolean equals(Object other)** is used to compare strings on the exact match their content. It returned false, if the strings do not match. It overrides the inherited method from Object. this method in detail will be discussed further.
- **boolean equalsIgnoreCase(String other)** – current string and string given as parameter are checked on matching without taking into account the register.
- **int compareTo(String other)** - current string and string given as parameter are checked by the lexicographical order. If the string is less than, returns -1 if it is greater then 1, otherwise (if they are equal), then 0.

Next, we consider the objects comparison in Java more generally.

In the book [4], in Section 5.2, devoted to the Object class was talked about the equals method. This method is inherited by all the classes and when it will be necessary to compare the current object, say Class A, with some object, is used this method. Semantics of comparision (what will be compared, with what and how) will be disclosed in this method, that override the inherited method. Test pattern, based on [4], is as in Fig. 15.5.

```

public boolean equals(Object o) {
    // a quick test to see if the objects are identical
    if (this == o) return true;
    // must return false if the explicit parameter is null
    if (o == null) return false;
    // if the classes don't match, they can't be equal
    if (getClass() != o.getClass()) return false;
    // now we know otherObject is a non-null Employee
    A other = (A)o;
    // test whether the fields have identical values
    return // checking the object fields;
}

```

Fig. 15.5. Example 4

Source: [4]

Object, that should be compared, say named `o` passed to the method by parameter of type `Object`.

Normally, for a particular class when this method will be overriding, only the fields comparison will be different. What will be compared with what and how will be defined here. Suppose some (comparing) object of class **A** has some other object of class **B** inside its physical boundaries (included in the composition relationship) as a part, referenced suppose by field **f**. Then we compare this part with the corresponding parts of the comparable object by:

```

...
& getFieldF().equals(o.getFieldF())
...

```

where `getFieldF()` returns the pointer to the object with the name of the **f**.

It is not allowed to compare parts, not included inside object's physical boundary. Usually do not necessarily compare their owner when comparing two computers, where purchased, etc. Comparing the two cars do not normally need to compare their data sheets, recording address etc., of course, if these data are not included in the physical boundary of the car. Comparison should be implemented based on the entity model, where should be clearly stipulated the physical boundary of the object.

In addition, comparison is based on the properties of equivalence relations in the mathematical sense:

- reflexivity property: `x.equals(x) == true;`
- symmetry property: `x.equals(y) <=> y.equals(x);`
- transitive property: `x.equals(y) & y.equals(z) => x.equals(z);`

- consistency property: different calls `x.equals(y)` return the same result if no change happened with the values of magnitude used in equals between the calls;
- nonemptiness property: `x.equals (null) == false`.

In [4] it is said that when it comes to the hierarchy of classes and it is necessary to compare the object of a class with the object of its subclass, in order to not violate the property of symmetry, it is necessary to determine the semantics of the comparison of the object of the subclass. If the semantics do not change, that is, the equivalence of the objects of the subclass is exactly the same as in the superclass, then the equals method of the superclass must use the instanceof predicate and must have a final qualifier. If the semantics changes, then the subclass overrides the equals method, and in the superclass, the `getClass()` method is used to check the identity of the classes.

The above comparison method can be referred to as object equivalence in the sense that an object compares itself with another object based on the semantics of its class equivalence. But we can talk about abstract (class) equivalence, where two objects are compared from some abstract point of view. For example, you can compare a pencil and a wooden stick to their length and thickness, a car and a capsule on the ferris wheel to account for the number of seats, etc. For this comparison, a class is created that corresponds to this abstraction, and in this class, the equals method will be implemented. This method compares objects in terms of the semantics of equivalence of this class. In addition, if special methods of accessing values are required for comparison, an interface is created that declares these access methods for comparison. The objects of any classes that implement this interface can be compared from the point of view of this class.

Example 5 from Fig. 15.6 shows both the object and abstract comparison.

```
interface ExistX {
    int getX();
}

class EqualByX {
    public static boolean equals (ExistX o1, ExistX o2) {
        return o1.getX() == o2.getX();
    }
}
```

```

class SuperClass implements ExistX {
    private int x;
    public SuperClass() {}
    public SuperClass(int x) {this.x = x;}
    public int getX() {return x;}
    public void setX(int x) {this.x = x;}
    public boolean equals(Object otherObject) {
        // a quick test to see if the objects are identical
        if (this == otherObject) return true;
        // must return false if the explicit parameter is null
        if (otherObject == null) return false;
        // if the classes don't match, they can't be equal
        if (!(otherObject instanceof SuperClass)) return false;
        // now we know otherObject is a non-null SuperClass object
        SuperClass other = (SuperClass) otherObject;
        // test whether the fields have identical values
        return x == other.x;
    }
    public static boolean equals (Object o1, Object o2) {
        if (!(o1 instanceof SuperClass)&&(o2 instanceof SuperClass)) return
false;
        SuperClass s1 = (SuperClass)o1;
        SuperClass s2 = (SuperClass)o2;
        return s1.getX() == s2.getX();
    }
};

class SubClass extends SuperClass {
    private int y;
    public SubClass() {}
    public SubClass(int y) {this.y = y;}
    public SubClass(int x, int y) { super(x); this.y = y;}
    public int getY() {return y;}
    public void setY(int y) {this.y = y;}
    public boolean equals(Object otherObject) {
        if (!super.equals(otherObject)) return false;
// super.equals checked that this and otherObject belong to the same class
        if (!(otherObject instanceof SubClass)) return false;
        // if (getClass() != otherObject.getClass()) return false;
        SubClass other = (SubClass) otherObject;
        return this.y == other.y;
    }
}

class Equals implements ExistX {
    private SuperClass a;
    private SubClass b;
    private int x;
    public Equals(int a, int b, int x) {
        this.a = new SuperClass(a);
        this.b = new SubClass(a, b);
        this.x = x;
    }

    public SuperClass getA(){return a;}
    public SubClass getB(){return b;}
    public int getX() {return x;}
    public static void main(String[] args) {

```

```

    Equals e = new Equals(10, 12, 10);
    System.out.println("---- Equalence of  a and b in object means ----");
    System.out.println("Object a is equal with object b:" +
e.getA().equals(e.getB()));
    System.out.println("Object b is equal with object a:" +
    e.getB().equals(e.getA()));
    System.out.println("---- Equalence of  a and b in object means ----");
    System.out.println("Object a is equal with object b in ExualByX means:" +
    EqualByX.equals(e.getA(), e.getB()));
    System.out.println("Object b is equal with object a in ExualByX means:" +
    EqualByX.equals(e.getB(), e.getA()));
    System.out.println("---- Equalence of  a and e in in ExualByX means ----");
    System.out.println("Object a is equal with object e in ExualByX means:" +
    EqualByX.equals(e.getA(), e));
    System.out.println("Object e is equal with object a in ExualByX means:" +
    EqualByX.equals(e, e.getA()));
    System.out.println("---- Equalence of a and e in static equals method ----
");
    System.out.println("Object a is equal with object e in ExualByX means:" +
    SuperClass.equals(e.getA(), e.getB()));
    System.out.println("Object e is equal with object a in ExualByX means:" +
    SuperClass.equals(e.getB(), e.getA()));
}
}

```

Fig. 15.6. Example 5

Source: [4]

The program from Fig. 15.6 gives the following result:

```

---- Equalence of  a and b in object means ----
Object a is equal with object b:true
Object b is equal with object a:false
---- Equalence of  a and b in object means ----
Object a is equal with object b in ExualByX means:true
Object b is equal with object a in ExualByX means:true
---- Equalence of  a and e in in ExualByX means ----
Object a is equal with object e in ExualByX means:true
Object e is equal with object a in ExualByX means:true
---- Equalence of  a and e in in static equals method ----
Object a is equal with object e in ExualByX means:true
Object e is equal with object a in ExualByX means:true

```

In the code from Fig. 15.6, it is declared one interface (**ExistX**) and classes **EqualByX**, **SuperClass**, **SubClass** and **Equals** are declared. Consider the process that is deployed in the **main** method of the **Equals** class.

An object **e** (of the **Equals** class) is created, where the objects of the **SuperClass** and **SubClass** classes (with the names **a** and **b**) are aggregated. In addition, there is also a field-value **x**. The received object scheme is shown in Fig. 15.7.

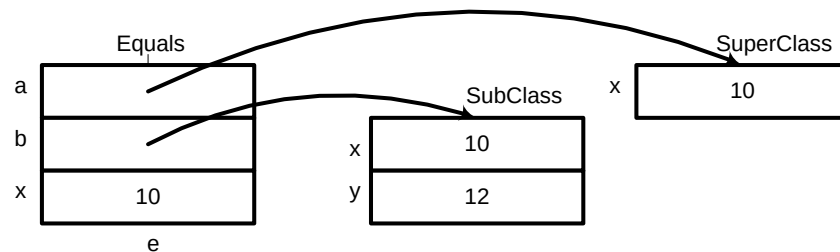


Fig. 15.7. Objects scheme, based on above program from. Fig. 15.6

Source: own work

By using `e.getA().equals(e.getB())`, the object that being indicated by the field **a** is compared with the object being indicated by the field **b**. Objects are compared based on the **SuperClass** class. In this class, the equals method of the **Object** class is overridden. In this case, is passed by parameter the object of its subclass **SubClass** for comparison. When comparing is used following algorithm:

1. If the addresses of the objects are equal, then the objects are equivalent
2. If the compared object is empty, the objects are not equivalent
3. If the type of the compared object is not polymorphic to **SuperClass**, then the objects are not equivalent
4. Otherwise, the compared object is treated as an **SuperClass** object and the values of the **x** field (the comparison semantics of the **SuperClass** class) are compared. Here does not use the exact match of classes to allow a polymorphic comparison of subclass objects.

This comparison returns an affirmative result after checking the 4th comparison item, that is, the compared objects are equivalent. Further, by using of `e.getB().equals(e.getA())`, seemingly symmetric comparison is performed - the object indicated by the field **b** is compared with the object indicated by the field **a**. The objects are compared on the basis of the **SubClass** class, which redefined the comparison method in the **SuperClass** class, where the comparison will be accomplished already by following algorithm:

1. If they are not equivalent by the equals method of the **SuperClass** class, then the objects are not equivalent
2. If the type of the compared object is not polymorphic to **SubClass**, then the objects are not equivalent. Here does not use the exact match of classes to allow a polymorphic comparison of subclass objects.
3. Otherwise the compared object is treated as **SubClass** and the values of the **y** field are compared (semantics of the **SubClass** class comparison).

Based on the values specified in the program, the objects are not equivalent, because the object of the **SuperClass** class, that passed by the parameter does not have a field with the name **y**. It should be reminded that these two relations are not the same, the first relation is the comparability of two objects based on the **SuperClass** class, the second is the comparability of two objects based on the **SubClass** class, so there can't be a symmetry here: **SuperClassEquality (a, b) ≠ SubClassEquality (A, b)**.

By means of **EqualByX.equals (e.getA (), e.getB ())**, the objects indicated by the fields **a** and **b**, respectively, are compared in an abstract manner from the point of view of the **EqualByX** class. This class can compare objects of type **ExistX**. Since both the **SuperClass** class and the **SubClass** class implement this interface, their objects can be compared with the comparison semantics of this class. Both objects have a field **x** and the value of this field is the same, so they are equivalent from this point of view. The following comparison - **EqualByX.equals (e.getB (), e.getA ())** is a symmetric comparison for the above comparison, both comparisons are performed on the basis of the **EqualByX** class and hence they will give the same result.

The following two comparisons are also abstract, based on **EqualByX**, but here the whole object (**e**) is compared to its part (**a**) using the comparison semantics of the **ExistX** class. From this point of view, both (direct and symmetric) comparisons will give an affirmative result.

In the last comparisons, the objects indicated by the fields **a** and **b** are abstractly compared from the point of view of the **SuperClass** class. This class except the standard method of comparing **equals**, inherited from **Object** and redefined in itself, has a static method of comparing **equals**, to which two objects of type **Object** are passed. The algorithm for comparing is following:

1. If the types of compared objects are not polymorphic **SuperClass**, then the objects are not equivalent
2. Otherwise, the objects are treated as **SuperClass** and the values of the field **x** (the comparison semantics of the **SuperClass** class) are compared. Here does not used the exact class match test to allow a polymorphic comparison of subclass objects.

These comparisons (direct and symmetrical) will give an affirmative result.

In the standard java library - **java.util.Arrays**, the static method **static boolean equals (type [] a, type [] b)** is implemented to compare arrays of primitive types, that is, type - **int, long, short, char, byte, boolean, float, double**).

Assignment and copying (cloning)

Assigning and copying objects are similar operations. In C ++, there is a default assignment where all the expanded parts of the object are copied, but you can override this operation, giving it different semantics. The semantics of assigning objects and passing parameters by value are identical. The default copy operation (the default copy constructor) and assignment have the identical meaning. You can also override the default copy constructor.

In Java, only the address of the object is copied when assigning. In the following fragment:

```
A original = new A();  
A copy = original;
```

the **original** and **copy** variables refer to the same object – Fig. 15.8.

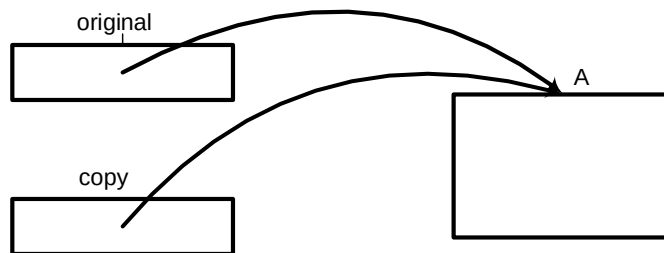


Fig. 15.8. Variable's pointers when simple copying

Source: own work

In order to copy the parts entering the physical boundary (clone), you must use the protected (**protected**) method `clone()` of the `Object` class or use serialization (linearization). Let's look at these approaches separately.

According to the standard JDK8 documentation [10], the clone method has a following signature:

```
protected Object clone() throws CloneNotSupportedException
```

To clone an object of a certain class, it is necessary that this class implements the `Cloneable` interface. This interface does not oblige the class to implement any method, it's just necessary to check that the formal implementation of this interface in the specified class (this check is performed in the `clone()` method) was successful.

The call to `x.clone()` creates a new copy (clone) of the specified object pointed to by the variable `x`, but the exact meaning of this cloning depends on the class of the object pointed to by `x`. According to the above documentation, when you override this method, the returning clone must be obtained by `super.clone()`.

Then the following expression is executed:

- `x.clone() != x. // This is provided by the Object class.`
- `x.clone().getClass() == x.getClass()`
- `x.clone().equals(x)`

In the documentation [10] and Java books, for example [4], is speak of deep and shallow cloning. In fact, in the process of construction, it is necessary to clone an object within its physical boundary.

In order to provide the independence of the cloned object from the source within the physical boundary of the object, in case of need, may be necessary to perform some "alteration" of the received object after `super.clone()`. If an object has a subobject that doesn't been included its physical boundary, then it is not necessary to do anything, otherwise, that is, if the subobject enters its physical boundary, then copy this part and set the pointer from the cloned object (Fig. 15.9).

In Fig. 15.9, in the original "original" object, rectangles with green represent parts of this object that are part of its physical boundary. These parts are indicated by a dashed green line. The yellow color indicates objects that are not part of its physical part. The result of cloning is indicated as "clone").

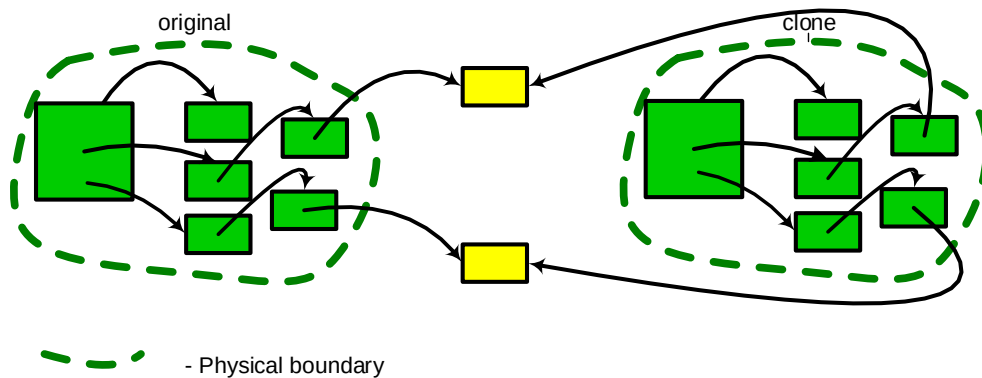


Fig. 15.9. Cloning of a complex object

Source: own work

The Example 6 (Fig. 15.10) shows cloning within the physical boundary.

```
package Examples;
public class ForClone implements Cloneable{
    private int intField;
    public ForClone (int i) {
        intField = i;
    }
}
```

```

    }
    public ForClone clone() throws CloneNotSupportedException {
        ForClone cloned = (ForClone)super.clone();
        return cloned;
    }
    public void setInt (int i) {intField = i;}
    public int getInt() {return intField;}
}

package Examples;
public class Cloning implements Cloneable{
    private double doubleField;
    private ForClone forField;
    public Cloning (int i, double d) {
        forField = new ForClone(i);
        doubleField = d;
    }
    public Cloning clone() throws CloneNotSupportedException {
        Cloning cloned = (Cloning)super.clone();
        cloned.forField = forField.clone();
        return cloned;
    }
    public static void main (String[] args) throws CloneNotSupportedException {
        Cloning sc = new Cloning(12, 12.12);
        Cloning scNew = (Cloning)sc.clone();
        scNew.doubleField = 24.24;
        scNew.forField.setInt(24);
        System.out.println("Original = "+sc.doubleField);
        System.out.println("Clone = "+scNew.doubleField);
        System.out.println("Original = "+sc.forField.getInt());
        System.out.println("Clone = "+scNew.forField.getInt());
    }
}

```

Fig. 15.10. Example 6

Source: [4]

The main (**main**) method is created (**sc**) and it is cloned (**scNew**). After cloning, the clone fields are corrected: with a simple **double** (**doubleField**) type and a **ForClone** type (**forField**). Since the **forField** field enters the physical boundary of this object, it is also cloned. To do this, the **clone** method of this object is called and he knows how to clone.

As you can see from the result, the adjustment affects only the clone, and the fields of the original object do not change their values:

```

Original = 12.12
Clone = 24.24
Original = 12
Clone = 24

```

When working with arrays, you can also apply `clone()` to arrays. In this case, the values of the array element are copied to the newly created array. Thus, if the elements of the array are of primitive type, then this primitive value is copied, if - the object (that is, the pointer) then the pointer is copied, thereby pointing to the same object from the original and cloned elements. As an example, the fragment from Fig. 15.11 can be cited: continuation of the main method.

```
Cloning[] scArr = new Cloning[4];
scArr[0]= sc;
scArr[1]= sc;
scArr[2]= sc;
scArr[3]= sc;
for (Cloning c : scArr) {
    System.out.println("Source: double = "+c.doubleField);
    System.out.println("Source: forField.int = "+c.forField.getInt());
}
Cloning[] scArrClone = scArr.clone();
scArrClone[1].doubleField = 36.36;
scArrClone[1].forField.setInt(36);
for (Cloning c : scArr) {
    System.out.println("Original: double = "+c.doubleField);
    System.out.println("Original: forField.int = "+c.forField.getInt());
}
for (Cloning c : scArrClone) {
    System.out.println("Clone: double = "+c.doubleField);
    System.out.println("Clone: forField.int = "+c.forField.getInt());
}
```

Fig. 15.11. Example 7

Source: [4]

Here, after cloning the array, the values of the 1st element of the cloned array are corrected. After that all elements of the original and cloned arrays are highlighted. As you can see from the result, the original array also changes:

```
Source: double = 12.12
Source: forField.int = 12
Source: double = 12.12
Source: forField.int = 12
Source: double = 12.12
Source: forField.int = 12
Source: double = 12.12
Source: forField.int = 12
Original: double = 36.36
Original: forField.int = 36
Original: double = 36.36
Original: forField.int = 36
Original: double = 36.36
Original: forField.int = 36
```

```

Original: double = 36.36
Original: forField.int = 36
Clone: double = 36.36
Clone: forField.int = 36
Clone: double = 36.36
Clone: forField.int = 36
Clone: double = 36.36
Clone: forField.int = 36
Clone: double = 36.36
Clone: forField.int = 36

```

When cloning arrays, the entire responsibility to "follow" the physical boundary lies on the class, that contains the array inside itself.

Another approach to cloning lies through linearization. When linearizing, the object is first converted to an array of bytes and will be re-created an object. The resulting object will be a clone of the source object. The class whose object is cloned (the enclosing object) and all classes whose objects enter the physical boundary of the enclosing object as part must implement the **Serializable** interface. This interface does not oblige the class to implement any method in the same way as the **Cloneable** interface, but the only difference is that when cloning it was necessary to override the **clone()** method, and when you linearize, you do not need to redefine anything. The Example 7 (Fig. 5.11) is realized by linearizing by code from Fig. 5.12.

```

package Examples;
import java.io.Serializable;

public class ForClone implements Serializable {
    private int intField;
    public ForClone (int i) {
        intField = i;
    }
    public void setInt (int i) {intField = i;}
    public int getInt() {return intField;}
}

package Examples;
import java.io.*;
import java.util.*;

public class Serializing implements Serializable{
    private double doubleField;
    private ForClone forField;
    public Serializing (int i, double d) {
        forField = new ForClone(i);
        doubleField = d;
    }
    // The Clone tool

```

```

public Object clone() {
    try {
        // save the object to a byte array
        ByteArrayOutputStream bout = new ByteArrayOutputStream();
        ObjectOutputStream out = new ObjectOutputStream(bout);
        out.writeObject(this);
        out.close();
        // read a clone of the object from the byte array
        ByteArrayInputStream bin = new ByteArrayInputStream(bout.toByteArray());
        ObjectInputStream in = new ObjectInputStream(bin);
        Object ret = in.readObject();
        in.close();
        return ret;
    } catch (Exception e) {
        return null;
    }
}

public static void main (String[] args) throws CloneNotSupportedException {
    Serializing sr = new Serializing(12, 12.12);
    Serializing srNew = (Serializing)sr.clone();
    srNew.doubleField = 24.24;
    srNew.forField.setInt(24);
    System.out.println("Original doubleField = "+sr.doubleField);
    System.out.println("Clone doubleField = "+srNew.doubleField);
    System.out.println("Original forField.intField = "+sr.forField.getInt());
    System.out.println("Clone forField.intField = "+srNew.forField.getInt());
}
}

```

Fig. 15.12. Example 7 modified

Source: [4]

The result is the same as in the previous example:

```

Original doubleField = 12.12
Clone doubleField = 24.24
Original forField.intField = 12
Clone forField.intField = 24

```

15.5. CONCLUSION

For Java, deleting composite objects is very easy because of the automatic garbage collection. When none of the pointers points to the object, the object is automatically deleted. We can say that for a Java programmer such a task does not arise at all.

REFERENCES

- [1] Booch G., Maksimchuk R., Engle M., Young B., Conallen j., and Houston K. (2007) *Object-Oriented Analysis and Design with Applications*. 3-d ed. Addison-Wesley, 720 p.
- [2] Fowler M. (2000) *A Brief Guide to the Standard Object Modeling Language (3rd Edition)*. Addison-Wesley, 181p.
- [3] Meyer B. (1997) *Object-Oriented Software Construction*. 2-ed., Prentice Hall, 770p.
- [4] Horstmann C. and Cornell G. (2012) *Core Java*. 9th –ed., Prentice Hall, Vol 1, 2.
- [5] Deitel P. and Deitel H. (2009) *Java™ for Programmers: Deitel® Developer Series*. Prentice Hall, 1200p.
- [6] Gilbert S. and McCarty B. (1998) *Object-Oriented Design in Java*. SAMS, 731p.
- [7] Connolly T. and Begg C. (2002) *Database Systems. A Practical Approach to Design, Implementation, and Management*. 3-d ed. Addison-Wesley, 1440p.
- [8] Чанышев А. Н. (1981) Курс лекций по древней философии: Учебное пособие для философских факультетов и отделений университетов. Moscow, Высшая школа (in Russian).
- [9] <http://www.adobe.com/devnet/actionscript/learning/oop-concepts/composition-and-aggregation.html>
- [10] *JDK Documentation*. <http://docs.oracle.com/javase/specs/jls/se8/jls8.pdf>